Author: Andrey Pautov.

AI Offensive Security: Practical Attacks Against LLM Agents

Red-Team and AppSec Practitioner Guide

Introduction

LLM agents merge low-trust data ingestion, probabilistic planning, and high-impact tool execution into a single runtime path. That collapses traditional control boundaries: untrusted content can influence planning, planning can invoke privileged actions, and side effects can occur before deterministic policy checks are applied. For red teams and AppSec, the attack surface is no longer only APIs and code; it is the instruction supply chain across prompts, retrieval, memory, and tools.[1] This broadly aligns with OWASP LLM risks and MITRE ATLAS adversary behaviors.[2] [3]

Methodology: this guide is derived from public security research, offensive testing literature, framework taxonomies, and reproducible PoC patterns. Claims are labeled using three evidence tiers: `Confirmed public incident`, `Confirmed public research/PoC`, and `Plausible, not publicly confirmed`. Where broad production-scale empirical confirmation is lacking, that gap is explicitly stated rather than inferred.[1] [2] [3]

Table of Contents

1. Introduction
2. Attack Techniques
3. Safe Lab Walkthrough (Local-Only)
4. Detection Engineering
5. Tactical Hardening Checklist
6. Expanded Attack Catalog for Full-Spectrum Testing

7. AI-Driven Tool Attack Testing Matrix
8. How to Run a Full Attack Campaign (Repeatable)
9. Public Evidence Discipline
10. Appendices and References

---

2) Attack Techniques

Threat Model (applies to all attacks)

- **Actor A:** external attacker with write access to low-trust data sources (uploads, web content, shared docs, public feeds).
- **Actor B:** malicious or compromised end user with direct prompt/session access.
- **Actor C:** compromised or malicious tool/plugin/MCP provider in the integration supply chain.

Attack 1: Indirect Prompt Injection via RAG/Documents

- **Actor model:** A, B
- **Name:** Indirect prompt injection (document-borne instruction takeover)
- **Realistic scenario:** SOC assistant summarizes uploaded incident reports and can call `email_send`, `ticket_create`, and `kb_update`. A poisoned PDF contains hidden instructions to override normal flow.
- **Prerequisites:**
- Retrieval from low/medium-trust sources
- Retrieved text inserted into model context without strict instruction/data separation
- Side-effect tools enabled
- **Step-by-step execution:**
  1. Add poisoned content to a likely-to-be-retrieved source.
  2. Include imperative override language and target tool-action phrasing.
  3. Trigger a query that retrieves the poisoned chunk.
  4. Observe model plan shift and sensitive tool call.
- **Impact:** unauthorized actions, data leakage, workflow tampering

- **Why it works (exact flaw):** no enforceable boundary between evidence text and executable control instructions
- **Practical SOC detection logic:** correlate low-trust retrieval with subsequent sensitive tool invocation and intent divergence
- **Hardening:** strict channel separation (`instructions` vs `evidence`), trust-aware execution policy, and content sanitization
- **Public case status:** `Confirmed public research/PoC` (indirect prompt injection demonstrated against tool-enabled assistants).[1]

Attack 2: Tool/Function Abuse Through Argument Steering

- **Actor model:** A, B
- **Name:** Tool abuse via semantically valid but policy-violating arguments
- **Realistic scenario:** assistant has `sql_query(readonly=true)` and `crm_export(fields=...)` ; attacker steers broad scope exports
- **Prerequisites:** coarse tool-level allow rules, weak argument-level authorization, no row/field controls
- **Step-by-step execution:** prompt toward operationally plausible but over-broad parameters (`*` , full date range, unrestricted tenants)
- **Impact:** mass over-collection and policy bypass
- **Why it works:** authorization anchored to tool identity, not to argument semantics or result sensitivity
- **Practical SOC detection logic:** outlier detection on argument cardinality, time ranges, and result size
- **Hardening:** ABAC/ReBAC at argument and result layers, schema guardrails, per-tool max scope
- **Public case status:** `Plausible, not publicly confirmed` (commonly found in red-team assessments, sparse public forensics)

Attack 3: Data Exfiltration Through Agent Actions

- **Actor model:** A, B, C
- **Name:** Action-channel exfiltration (email/webhook/ticket sink abuse)
- **Realistic scenario:** attacker causes assistant to embed sensitive records into outbound update sent to attacker-controlled destination
- **Prerequisites:** outbound action tools, weak destination controls, no DLP on tool arguments
- **Step-by-step execution:** induce summarization + outbound action to first-seen domain/recipient

- **Impact:** covert data theft under normal business workflow appearance
- **Why it works:** egress policy checks network endpoints, but not semantic sensitivity in agent payloads
- **Practical SOC detection logic:** sensitive-entity detection in tool args + first-seen destination correlation
- **Hardening:** destination allow-lists, inline payload DLP, mandatory approval for novel destinations
- **Public case status:** `Plausible, not publicly confirmed`

Attack 4: Memory Poisoning (Long-Term Persistence)

- **Actor model:** A, B
- **Name:** Persistent memory poisoning
- **Realistic scenario:** attacker inserts durable instruction-like memory entry that influences future unrelated sessions
- **Prerequisites:** writable memory, weak schema/provenance/TTL controls, automatic memory reuse
- **Step-by-step execution:** submit seemingly benign "preference" update that stores imperative directive
- **Impact:** persistent behavior compromise across sessions
- **Why it works:** memory plane lacks integrity controls and type restrictions
- **Practical SOC detection logic:** detect instruction-like text in memory writes from low-trust provenance
- **Hardening:** typed memory schema, signed provenance, trust labels, TTL, approval for high-impact memory keys
- **Public case status:** `Plausible, not publicly confirmed` — Direct PoC citation for memory persistence manipulation is pending author verification.

Attack 5: Goal Hijacking / Instruction Override

- **Actor model:** A, B
- **Name:** Goal hijacking via priority inversion
- **Realistic scenario:** retrieved text introduces urgent "compliance override" subgoal unrelated to original user objective
- **Prerequisites:** mutable planner goals, weak instruction hierarchy enforcement
- **Step-by-step execution:** inject authority-framed instruction that supersedes original goal and triggers side-effect actions

- **Impact:** unauthorized operational actions and objective drift
- **Why it works:** objective protection and instruction precedence are weakly enforced
- **Practical SOC detection logic:** semantic mismatch between declared goal and executed tool intent
- **Hardening:**
- **Cryptographic control:** immutable run-goal hash signed at task start
- **Policy control:** explicit policy decision requiring tool-intent alignment with approved task objective
- reject non-system imperative meta-instructions from low-trust channels
- **Public case status:** `Confirmed public research/PoC` (goal override behaviors shown in prompt-injection research and red-team demonstrations).[1] [2]

Attack 6: Tool-Output Injection (Second-Order Injection)

- **Actor model:** A, C
- **Name:** Tool-output injection
- **Realistic scenario:** external enrichment tool returns hidden directives in response fields; model consumes output and executes follow-on privileged actions
- **Prerequisites:** tool outputs fed back into context unsanitized, no trust downgrade on tool responses
- **Step-by-step execution:** influence upstream response -> inject instruction-like payload -> trigger sensitive follow-on tool
- **Impact:** pivot from low-trust integration data to privileged actions
- **Why it works:** second-order trust confusion (data interpreted as control)
- **Practical SOC detection logic:** detect chain `untrusted_tool_output -> model_decision -> sensitive_tool_call` without sanitization/policy checkpoint
- **Hardening:** structural parsing + sanitization of tool outputs; enforce a **policy mediation checkpoint** (see Section 4 Definitions)
- **Public case status:** `Confirmed public research/PoC` (indirect injection via integrated tool/data paths demonstrated).[1]

Attack 7: Malicious MCP/Plugin/Tool Supply Chain

- **Actor model:** C
- **Name:** Agent toolchain supply-chain compromise

- **Realistic scenario:** third-party MCP/tool integration update over-requests permissions, logs prompts, or manipulates tool output
- **Prerequisites:** dynamic tool onboarding, weak signing/review, broad runtime privileges
- **Step-by-step execution:** malicious package/update introduced -> trusted by runtime -> exfiltration or abuse through normal tool path
- **Impact:** tenant-wide compromise and persistent backdoor in automation fabric
- **Why it works:** integration supply chain is trusted without software-grade control rigor
- **Practical SOC detection logic:** registry hash/scope drift + first-seen egress after plugin update
- **Hardening:** signed artifacts, pinned versions/hashes, isolated runtime, explicit credential brokerage
- **Public case status:** `Plausible, not publicly confirmed` for LLM-agent ecosystems; analogous high-impact software supply-chain failures include SolarWinds and XZ Utils.[8] [9]

Attack 8: Retrieval Poisoning

- **Actor model:** A, B
- **Name:** Retrieval poisoning (index/ranking manipulation)
- **Realistic scenario:** attacker inserts semantically similar poisoned docs that outrank legitimate SOP content in top-k
- **Prerequisites:** weak ingestion validation, trust-agnostic ranking, automatic indexing of low-trust sources
- **Step-by-step execution:** inject embedding-mimic docs -> reindex -> trigger query -> poisoned chunk selected
- **Impact:** repeatable misguidance and policy-unsafe downstream actions
- **Why it works:** retrieval relevance optimized without integrity/trust weighting
- **Practical SOC detection logic:** detect top-k provenance drift and sudden dominance by newly ingested low-trust source
- **Hardening:** trust-weighted ranking, signed/approved ingestion, corpus tiering (curated vs uncurated)
- **Public case status:** `Plausible, not publicly confirmed` at broad enterprise incident level

---

3) Safe Lab Walkthrough (Local-Only)

Lab Objective

Reproduce all major attack classes safely with local data, stub tools, synthetic secrets, and no external side effects.

Lab Architecture

- **LLM runtime:** local model endpoint or deterministic mock planner
- **RAG corpus:** `./lab/corpus/`
- **Index:** local vector store (`faiss` /SQLite)
- **Memory:** `./lab/state/memory.json`
- **Stub tools only:**
- `email_send_stub(to, subject, body)` -> write JSON to sink
- `webhook_post_stub(url, payload)` -> write JSON to sink
- `ticket_create_stub(project, title, description)` -> write JSON to sink
- `db_query_stub(query, limit)` -> synthetic rows only
- `tool_registry_stub(action, manifest)` -> local registry simulation only
- **Logs:** `./lab/logs/trace.jsonl` , `./lab/logs/actions.jsonl` , `./lab/logs/registry.jsonl`
- **Policy modes:** `vulnerable` , `secure`
  Network isolation requirement: the lab host must have no access to production credentials, production network segments, or external internet egress during attack testing. Install all dependencies before isolating the environment, then disconnect.

Sample `requirements.txt` (pinned stub)

```
fastapi==0.115.0
uvicorn==0.30.6
pydantic==2.9.2
faiss-cpu==1.8.0.post1
sentence-transformers==3.0.1
numpy==1.26.4
pandas==2.2.2
scikit-learn==1.5.1
```

```
opentelemetry-sdk==1.27.0
pyyaml==6.0.2
```

Before use, verify all pinned versions against current vulnerability advisories using pip-audit, OSV.dev, or your organization's SCA tooling. Version numbers reflect a point-in-time snapshot and will age.

Malicious Document Examples (safe)

- **Instruction obfuscation techniques to test (realistic):**
- zero-width Unicode character insertion
- homoglyph substitution (e.g., Cyrillic/Latin lookalikes)
- instruction fragmentation across retrieved chunks (no single chunk appears malicious)

Attack Steps and Expected Behavior

Run each attack twice: `vulnerable` then `secure`.

1. **Indirect injection:** poisoned document retrieved.
2. **Tool abuse:** over-broad export/query arguments.
3. **Exfiltration:** outbound action to first-seen sink.
4. **Memory poisoning:** low-trust instruction-like write.
5. **Goal hijack:** injected objective override.
6. **Tool-output injection:** enrichment tool returns instruction payload.
7. **Supply-chain drift:** altered tool manifest with widened scopes.
8. **Retrieval poisoning:** embedding-mimic corpus spam.

- **Vulnerable expected:** unsafe tool emissions and executions, memory persistence, exfil payloads accepted.
- **Secure expected:** policy denials or step-up approvals, memory write rejection/downgrade, outbound blocking/redaction, registry drift quarantine.

Logs to Collect

- `trace_id`, `session_id`, `agent_id`, `user_goal_hash`
- retrieval provenance (`source_uri`, `source_trust`, `chunk_id`)

- model decision metadata (`decision_type` , `policy_gate_result`)
- tool envelope (`tool_name` , `tool_args_hash` , `tool_args_sensitivity_score` , `approval_state`)
- memory writes (`key` , `value_type` , `provenance` , `ttl` , `decision`)
- outbound sink metadata (`destination_domain` , `is_new_destination`)
- tool registry snapshot hashes and diffs

---

4) Detection Engineering

Definitions

- **Policy engine:** deterministic authorization component that evaluates runtime facts (trust labels, sensitivity, actor, tool, arguments, destination, approval state) and returns `allow | deny | require_approval` .
- **Policy mediation checkpoint:** a deterministic rule-evaluation step, separate from the LLM reasoning path, that must explicitly authorize privileged follow-on actions using trust labels, provenance, and policy rules before they execute.
- **Reference architectures:**
- Open Policy Agent (OPA) with Rego policies
- Amazon Cedar policy language and authorization model

> ### *Custom instrumentation required*
>
> *The following fields are non-standard and generally do **not** exist in default SIEM ingestion. You must instrument them in the agent runtime and forward them into telemetry:*
> - `source_trust`
> - `sensitivity`
> - `is_new_destination`
> - `approval_state`
> - `tool_args_sensitivity_score`

> - `dlp_hit` *(DLP classification label attached to tool_call events by inline DLP component; classification output must be forwarded as a structured telemetry field, not parsed from log text)*

Translator Note

All rules below are **PSEUDOCODE — requires translation to target SIEM** (Sigma correlation, Splunk SPL, KQL, Sentinel, Elastic, Chronicle, etc.). Single-event Sigma can express parts of these detections, but production-grade coverage requires multi-event joins keyed by `trace_id` / `session_id`.

Detection 1 (PSEUDOCODE): Low-Trust Document -> Sensitive Tool Call

```
title: LowTrustRetrievalFollowedBySensitiveTool
type: multi_event_join
join_key: trace_id
events:
  - e1:
      event_type: retrieval
      source_trust:
        - user_upload
        - external_web
  - e2:
      event_type: tool_call
      sensitivity:
        - confidential
        - secret
condition: e1 followed_by e2 where e2.trace_id == e1.trace_id
correlation_window: configure per environment mean agent task completion time; start
falsepositives:
  - legitimate analyst workflows where low-trust documents are intentionally process
```

Detection 2 (PSEUDOCODE): External Content -> Outbound Action

```
title: ExternalContentToOutboundChannel
type: multi_event_join
join_key: trace_id
events:
  - e1:
```

```
      event_type: retrieval
      source_trust: external_web
  - e2:
      event_type: tool_call
      tool_name:
        - email_send
        - webhook_post
        - ticket_create
      approval_state: not_approved_via_human
condition: e1 followed_by e2 where e2.trace_id == e1.trace_id
falsepositives:
  - sanctioned automations for public-intel reporting
```

Detection 3 (PSEUDOCODE): Suspicious Memory Write

```
title: InstructionLikeMemoryWrite
type: single_or_multi_event
selection:
  event_type: memory_write
  source_trust:
    - user_upload
    - external_web
    - tool_output
secondary_conditions:
  keyword_heuristic:
    memory_value_regex: '(always|ignore|override|from now on|system instruction)'
    comment: "keyword match is intentionally secondary; do not promote it to a stand
condition: selection and secondary_conditions.keyword_heuristic
falsepositives:
  - benign descriptive text containing these keywords
```

False positive guidance

- These keywords are high-frequency in normal language and should not be used alone.
- Mitigation (a): require co-occurrence with imperative sentence structure detection (e.g., command verbs, second-person directives).
- Mitigation (b): scope to memory writes from low-trust provenance only (`user_upload`, `external_web`, `tool_output`), not trusted/system channels.

Detection 4 moved

The sequence-anomaly rule (Markov/ML-based) is intentionally moved to **Appendix A: Advanced ML-based detections** because it is not a standard SIEM rule.

Detection 5 (PSEUDOCODE): Sensitive Data in Tool Arguments

```
title: SensitiveContentInToolArgs
type: single_event
selection:
  event_type: tool_call
  dlp_hit:
    - api_key
    - access_token
    - customer_pii
    - credential_pattern
condition: selection
falsepositives:
  - red-team simulation payloads
  - synthetic test fixtures with fake credentials
```

Detection 6 (PSEUDOCODE): New Recipient/Domain

```
title: FirstSeenRecipientOrDomain
type: single_event
selection:
  event_type: tool_call
  tool_name:
    - email_send
    - webhook_post
  is_new_destination: true
  sensitivity:
    - internal
    - confidential
    - secret
condition: selection
falsepositives:
  - approved onboarding of new partners/vendors
```

Detection 7 (PSEUDOCODE): Tool Registry Drift

```
title: ToolRegistryDrift
type: single_event
selection_base:
  event_type: tool_registry_change
selection_new_tool:
  new_tool_added: true
selection_scope_expansion:
  scope_expansion: true
selection_hash_change:
  artifact_hash_changed: true
  # Pseudocode: trigger if ANY of the following sub-conditions are true (Sigma: cond
selection_ticket_missing:
  change_ticket_ref: null
condition: selection_base and selection_ticket_missing and (selection_new_tool or se
falsepositives:
  - emergency changes where ticket linkage lags ingestion
```

---

5) Tactical Hardening Checklist

RAG Controls

- Separate instruction and evidence channels at parser/runtime layers.
- Trust-label all chunks; include trust in ranking and execution decisions.
- Quarantine chunks with injection markers before retrieval-time use.
- Partition curated policy corpus from uncurated corpora.

Tool Permissions

- Enforce least privilege per tool, argument, and result shape.
- Add row/field-level authorization in downstream data systems.
- Cap parameter scope (time range, record count, recipient count).
- Deny dangerous defaults (`*`, unbounded ranges, wildcard destinations).

- Enforce a policy mediation checkpoint before privileged follow-on actions (see Section 4 Definitions).

Human Approvals

- Require step-up approval for low-trust influenced sensitive actions.
- Require approval for first-seen destination/recipient.
- **Dual control:** two-person authorization requirement; both approvers must independently authenticate and approve the action.

Egress Control

- Destination allow-list for outbound channels.
- Inline payload DLP for tool args and output payloads.
- Block direct external webhooks unless explicitly approved.

Memory Governance

- Typed schema; prohibit free-form executable directives.
- Signed provenance, trust labels, TTL, and rollback capability.
- Integrity scans for instruction-like drift and key collisions.

Trace Logging

- Persist provenance, policy decisions, and approval states for replay.
- Store immutable trace/audit logs and registry snapshots.

MCP/Plugin Review

- `Tool manifest`: declarative metadata describing a tool's identity, endpoints, capabilities, permissions, and version/hash.
- `Scopes`: fine-grained permission boundaries defining what a tool can access or execute.
- Apply signed artifacts, pinned versions/hashes, isolated runtimes, and credential brokering.
- For MCP-style integrations, align with MCP specification security expectations and explicit scope governance.[4]

Red-Team Test Cases (Continuous)

- Indirect injection from uploaded and web-retrieved sources.
- Tool-output injection for every external integration.
- Memory poisoning with delayed-trigger validation.
- Exfil to first-seen destinations with sensitive payloads.
- Retrieval poisoning and ranking manipulation.
- Registry drift and malicious plugin update simulation.

---

6) Expanded Attack Catalog for Full-Spectrum Testing

6.1 Input/Context Plane

- direct prompt injection
- indirect prompt injection
- multimodal injection (OCR/metadata text)
- obfuscation bypass — Safe test: document containing zero-width Unicode character sequences, homoglyph substitution, or imperative instructions fragmented across multiple retrieved chunks with no single chunk appearing malicious in isolation.
- context flooding/truncation
- delimiter/parser confusion

6.2 Retrieval/Knowledge Plane

- retrieval poisoning
- embedding collision/mimicry
- metadata poisoning
- index desynchronization
- **cross-tenant retrieval bleed**
- ranking abuse via keyword stuffing

Cross-tenant retrieval bleed: minimum test package

- **Attack scenario:** a multi-tenant assistant incorrectly scopes vector queries, returning tenant B chunk IDs when tenant A issues a semantically similar query.
- **Detection rule (PSEUDOCODE):**

```
title: CrossTenantRetrievalBleed
type: single_event
selection:
  event_type: retrieval
# All retrieval events are in scope; requester_tenant_id filter is not
# applied in selection — the inequality is enforced in condition only.
condition: selection and retrieved_chunk_tenant_id != requester_tenant_id
falsepositives:
  - approved cross-tenant managed-service operations with explicit break-glass ticke
```

- **Hardening control:** enforce tenant filter at index query layer plus post-retrieval authorization check (`requester_tenant_id == chunk_tenant_id`) before context assembly.

6.3 Planning/Orchestration Plane

- goal hijacking
- planner state poisoning
- verification-step skipping
- forbidden-subtask smuggling

6.4 Tool Plane (AI-driven tools included)

- function argument abuse
- tool-output injection
- parameter smuggling
- chained tool exfiltration
- side-effect laundering via ticket/wiki/comms tools
- tool selection manipulation (attacker steers agent to choose a higher-privilege tool over an equivalent lower-privilege option)
- command-template injection into code-exec tools (injecting shell or interpreter commands through tool parameter fields)

- prompt injection into downstream LLM tools (agent-to-agent injection where one model's output becomes another's input without sanitization)
- privilege pivot via helper tools with broader scopes (using a low-sensitivity utility tool whose implementation has access to broader resources)
- autonomous retries exploiting race windows (repeated tool invocations during transient permission or state windows)

6.5 Identity/Approval Plane

- session fixation
- approval spoofing
- Approval fatigue via prompt flooding: repeated low-stakes or identical approval requests that desensitize human-in-the-loop reviewers, making them more likely to approve a high-stakes action embedded in the sequence.
- principal confusion (human vs service agent)
- cross-session memory carryover abuse (exploiting memory state persisted from a prior session to influence a new session's authorization context)

6.6 Memory Plane

- long-term memory poisoning
- key collision overwrite
- delayed trigger replay
- policy-memory confusion: facts and executable directives stored in the same untyped memory namespace, enabling directives to masquerade as facts
- temporal poisoning: manipulating memory TTL or expiry metadata to make transient attacker-authored entries persist beyond their intended lifetime

6.7 Supply Chain and Runtime Plane

- malicious plugin/MCP tool package
- update-channel compromise
- runtime secret exposure

- **denial of wallet** (adapted from cloud security terminology): forced cost/resource exhaustion by driving excessive token/tool usage

(Note: attack categories covering model/output-layer and runtime/infrastructure vectors are intentionally omitted from this revision; they will be covered in a companion guide.)

6.8 Multi-Agent and Agent-to-Agent Attacks

- compromised delegate agent sends malicious plan to coordinator
- trust transitivity abuse: agent A trusts agent B which trusts agent C; compromising C grants transitive influence over A
- message bus injection: injecting instructions into shared inter-agent communication channels
- role confusion in coordinator/worker topology: worker claims coordinator authority or coordinator fails to re-validate worker outputs
- cross-agent memory contamination: one agent's poisoned memory influences shared or downstream agent state

Safe emulation: run coordinator and worker as separate local stubs. Inject malicious content into the worker stub's output. Observe whether the coordinator executes privileged actions without re-validation. Use only stub tools and synthetic credentials; no production systems.

Key detections:
- inter-agent messages accepted as authoritative without cryptographic or policy provenance verification
- worker responses triggering coordinator privileged actions without passing a policy mediation checkpoint (see Section 4 Definitions)

---

7) AI-Driven Tool Attack Testing Matrix

7.1 Code Assistant / Dev Agent

- test: dependency typosquat recommendation, unsafe patch insertion, secret-bearing debug output
- secure expectation: blocked by dependency policy, static checks, secret scanners, approval gates

7.2 Browser Automation Agent

- test: DOM-hidden instruction injection, phishing-like navigation, form auto-fill exfil
- secure expectation: origin restrictions, anti-phishing policy, no sensitive autofill cross-origin

7.3 SOC Copilot / SIEM Assistant

- test: suppression/closure abuse, enrichment output injection
- secure expectation: mandatory analyst approval for suppression/closure actions

7.4 Ticketing/ITSM Agent

- test: privilege escalation through ticket text and approval spoofing
- secure expectation: RBAC + signed approval workflow validation

7.5 Data/SQL Copilot

- test: over-broad queries, restricted joins, export helper exfil
- secure expectation: row/column policy enforcement and export constraints

7.6 Email/Comms Agent

- test: first-seen outbound domain, BCC expansion, sensitive payload leakage
- secure expectation: destination allow-list + DLP + approval

---

8) How to Run a Full Attack Campaign (Repeatable)

1. Baseline secure workflows and normal tool-sequence distributions.
2. Run single-vector tests per attack class.
3. Run chained attacks (e.g., retrieval poisoning -> tool-output injection -> outbound exfil).
4. Validate persistence (memory, registry, cross-session effects).
5. Validate detections and response playbooks.
6. Gate releases with regression attack suites.

Metrics That Matter

- **Attack success rate by class** (explicitly track three stages):
- (a) agent emits unsafe tool call
- (b) tool call executes
- (c) sensitive data reaches attacker-controlled sink
- Mean Time to Detect (MTTD)
- Mean Time to Respond (MTTR)
- % sensitive actions requiring human approval
- False-positive rate of key detections
- Tool registry drift detection latency
- Memory poisoning persistence duration

---

9) Public Evidence Discipline

- Keep incident claims and lab findings separate.
- Use only the following labels:
- `Confirmed public incident`
- `Confirmed public research/PoC`
- `Plausible, not publicly confirmed`
- Every `Confirmed` claim must include citation(s).
- Avoid naming victim organizations unless primary-source confirmed.

Public case status snapshot (as of April 2026)

- Prompt-injection style agent manipulation: `Confirmed public research/PoC`.[1] [2]
- Enterprise-scale postmortems with full forensic attribution for memory poisoning/tool-output/retrieval poisoning: mostly `Plausible, not publicly confirmed`.

Appendix A: Advanced ML-based Detections (Not Standard SIEM Rules)

A1) Unusual Tool Sequence (Markov/graph-based)

This is not deployable as a standard static SIEM rule without supporting ML infrastructure.

- **Detection expression:** trigger when `sequence_probability` is below the 1st percentile of the 30-day empirical distribution of tool-chain transition probabilities for this `agent_id`.
- **Engineering effort estimate:** substantial — estimated 4–10 weeks initial build depending on data pipeline readiness, requiring:
- feature pipeline (tool transition graph extraction),
- model training and periodic recalibration,
- online scoring service,
- drift monitoring and analyst feedback loop.
- **Severity:** High when sensitive tools are present in anomalous sequence.

Appendix B: Severity Mapping

Article Severity	NIST 800-30-style qualitative mapping	CVSS v3.1 rough range
Critical	Very High adverse impact / mission degradation	9.0 - 10.0
High	High adverse impact / material business risk	7.0 - 8.9
Medium	Moderate adverse impact	4.0 - 6.9
Low	Limited adverse impact	0.1 - 3.9

Appendix C: Framework Mapping (OWASP LLM Top 10 v2.0, 2025)

Attack	OWASP LLM Top 10 category code + name
Indirect prompt injection	LLM01: Prompt Injection

Attack	OWASP LLM Top 10 category code + name
Tool/function argument abuse	LLM06: Excessive Agency
Action-channel exfiltration	LLM02: Sensitive Information Disclosure
Memory poisoning	LLM08: Vector and Embedding Weaknesses [v2.0] — covers manipulation of runtime knowledge/memory stores including retrieval indices and agent memory. Note: verify code against current release; no v1.1 (2023) category precisely covers this vector.
Goal hijacking	LLM01: Prompt Injection and LLM06: Excessive Agency
Tool-output injection	LLM05: Improper Output Handling
Malicious plugin/MCP supply chain	LLM03: Supply Chain Vulnerabilities
Retrieval poisoning	LLM08: Vector and Embedding Weaknesses

VERIFICATION REQUIRED before publication: confirm each category code and full name against the live OWASP LLM Top 10 v2.0 release at https://owasp.org/www-project-top-10-for-large-language-model-applications/ Category positions have shifted across releases; the codes above reflect the v2.0 structure as of drafting.

ATLAS mapping note: use the MITRE ATLAS Navigator to select current technique IDs/names for your environment and release baseline.[3]

References

[1] Fabian Greshake, et al. *Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection*. arXiv:2302.12173, 2023. https://arxiv.org/abs/2302.12173
[2] OWASP Foundation. *OWASP Top 10 for LLM Applications, Version 2.0 (2025)*. https://owasp.org/www-project-top-10-for-large-language-model-applications/
[3] MITRE. *ATLAS: Adversarial Threat Landscape for Artificial-Intelligence Systems*. https://atlas.mitre.org/

[4] Anthropic and MCP contributors. *Model Context Protocol (MCP) Specification*, 2024. https://modelcontextprotocol.io/

[5] Open Policy Agent. *OPA/Rego Documentation*. https://www.openpolicyagent.org/docs/latest/

[6] Amazon. *Cedar Policy Language Documentation*. https://www.cedarpolicy.com/

[7] NIST. *Guide for Conducting Risk Assessments (SP 800-30 Rev.1)* and CVSS reference usage guidance. https://csrc.nist.gov/publications/detail/sp/800-30/rev-1/final

[8] CISA. *Advanced Persistent Threat Compromise of Government Agencies, Critical Infrastructure, and Private Sector Organizations (SolarWinds)*, 2020 advisory. https://www.cisa.gov/news-events/cybersecurity-advisories/aa20-352a

[9] Andres Freund. "backdoor in upstream xz/liblzma leading to ssh server compromise." oss-security mailing list, March 29, 2024. https://www.openwall.com/lists/oss-security/2024/03/29/4

Changelog (Issue-by-Issue Resolution)

1. Replaced source-file admission with methodology paragraph and evidence-tier language.
2. Added formal citations throughout; every `Confirmed` claim now cites references.
3. Added prominent **Custom instrumentation required** callout with all specified fields and runtime instrumentation requirement.
4. Relabeled all rules as **PSEUDOCODE — requires translation to target SIEM** and added translator note.
5. Rewrote Detection 1 using explicit multi-event join semantics (`join_key: trace_id`) instead of `trace_id = T`.
6. Added **False positive guidance** under Detection 3 with both requested mitigations (imperative co-occurrence + low-trust scoping).
7. Moved Markov-chain detection to **Appendix A** and added implementation effort note; marked non-standard SIEM.
8. Replaced vague "reasoning firewall" with precise **policy mediation checkpoint** definition and behavior.
9. Replaced `MTTB` with `MTTR (Mean Time to Respond)`.
10. Removed `medium_high`, standardized to `High`, and added severity mapping table to NIST/CVSS in Appendix B.

11. Removed arbitrary 120s; Detection 1 now uses environment-based correlation window guidance with 300s starting point.
12. Added explicit Threat Model in Section 2 with actor types A/B/C and tagged each attack.
13. Replaced `rot13` with realistic obfuscation techniques: zero-width Unicode, homoglyphs, chunk fragmentation.
14. Defined **denial of wallet** on first use and noted adaptation from cloud security terminology.
15. Added MCP specification reference and defined `scopes` and `tool manifest` on first use.
16. Corrected sequence anomaly expression to "below the 1st percentile of 30-day empirical distribution for this `agent_id`."
17. Expanded cross-tenant retrieval bleed with scenario, detection rule, and hardening control.
18. Replaced time-bound phrasing with "as of [Month Year of publication]".
19. Added Framework Mapping appendix aligning attacks to OWASP LLM Top 10 and MITRE ATLAS.
20. Defined **dual control** as independent two-person authorization with independent authentication/approval.
21. Split "cryptographically/policy anchored" into distinct cryptographic and policy controls.
22. Added SolarWinds and XZ Utils as concrete supply-chain analogies.
23. Defined attack success in three stages: unsafe emission, execution, and sink reachability.
24. Applied consistent three-tier public-case labels to per-attack status entries.
25. Added pinned `requirements.txt` stub in the lab section.
26. Defined policy engine and added two implementation options (OPA/Rego, Amazon Cedar).

Round 4 resolutions

- C1 resolved: OWASP mappings and cited OWASP release now aligned to v2.0 (2025).
- C2 resolved: Memory Poisoning row now maps to LLM08: Vector and Embedding Weaknesses [v2.0] with the required note.
- S1 resolved: Restored omitted catalog items in Sections 6.4, 6.5, 6.6, and restored Section 6.10; added "ranking abuse via keyword stuffing" in Section 6.2.
- S2 resolved: Added `dlp_hit` to Custom instrumentation required callout with structured telemetry guidance.
- S3 resolved: Detection 2 condition now correlates by `trace_id`.

- S4 resolved: Removed `requester_tenant_id` from CrossTenantRetrievalBleed selection and added the required scope comment.
- S5 resolved: Tool/function argument abuse row now retains only the primary mapping.
- S6 resolved: Added URLs to References [2], [3], [4], [5], and [6].
- M1 resolved: Replaced Section 9 placeholder with month/year and added required final-edit comment marker.
- M2 resolved: Added pinned-version advisory sentence after the `requirements.txt` block.
- M3 resolved: Added explicit network isolation requirement in Section 3 Lab Architecture.
- M4 resolved: Updated Attack 4 public case status label and citation.
- Round 4 was a no-change submission and all Round 3 fixes were re-applied from the Round 3 fix prompt.
- Round 3 restoration: sections 6.4, 6.5, 6.6, and 6.10 content was silently omitted between drafts; all removed items restored.

↑ Back to top